



Plant Guardian

Aplicación móvil para el cuidado inteligente de plantas de exterior e interior

1er informe de progreso

08/04/2026

Fiorella Lucia Queirolo Chavez

Índice

1. Introducción.....	3
2. Planificación.....	3
2.1. Primer sprint.....	3
2.2. Segundo sprint.....	4
3. Estado del proyecto.....	5
3.1. Documento de especificación del sistema.....	6
3.1.1. Cambios en los diagramas.....	6
3.1.2. Estructura de la base de datos.....	7
3.1.3. Arquitectura del sistema.....	8
3.2. Definición de las funcionalidades.....	8
3.2.1. Entrevistas a usuarios.....	8
3.2.2. Diseño de la interfaz.....	9
3.3. Construcción del backend.....	9
3.3.1. Agregación de las APIs.....	10
3.3.2. Gamificación.....	10
3.3.3. Librerías.....	11
3.4. Base de datos.....	11
4. Inteligencia artificial.....	12
5. Webgrafía.....	13

1. Introducción

El presente documento recoge el avance correspondiente a las primeras fases del desarrollo del proyecto Plant Guardian, siguiendo la planificación y metodología previamente definidas en el plan del proyecto.

En este informe se describe el estado actual del sistema, analizando el grado de cumplimiento de los objetivos establecidos para cada sprint, así como las tareas completadas y aquellas que aún se encuentran pendientes.

Asimismo, se presenta una síntesis del trabajo desarrollado en las distintas áreas del proyecto, incluyendo la definición de la lógica del sistema a través del documento de especificación, el diseño de la interfaz de usuario y la implementación del backend.

Por último, se hace referencia al uso de la inteligencia artificial como herramienta de apoyo durante el desarrollo del proyecto, ya que permitió resolver diversas dudas y facilitó significativamente el proceso de trabajo.

2. Planificación

Para controlar el seguimiento de las tareas, se utiliza la plataforma Trello en donde se definen las tareas a partir del diagrama WBS junto con el diagrama de Gantt, y se asignan al *sprint* correspondiente mediante etiquetas. Todo esto se realiza siguiendo la metodología indicada en la planificación inicial, la cual es Scrum.

El flujo de trabajo consiste en crear inicialmente las tareas en el *backlog*. Cuando comienza un *sprint*, las tareas planificadas se trasladan a la sección correspondiente. A medida que se empieza a trabajar en ellas, se mueven a la columna de “En desarrollo” y, una vez finalizadas, pasan a la sección de “Hecho”.

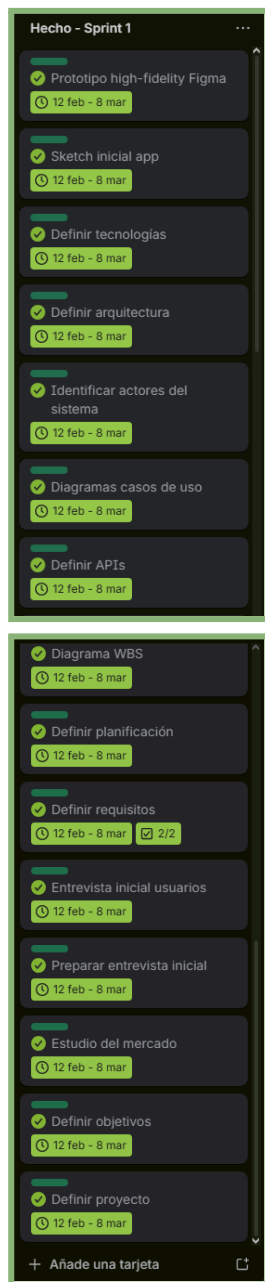
En caso de que una tarea incluya subtareas, estas se organizan mediante *checklists*, lo que permite un mayor control del progreso. Además, las tareas se agrupan por *sprints*, facilitando así una visión global del estado del proyecto. Por último, si una tarea ya completada requiere modificaciones posteriores, ésta se devuelve a la fase de “En desarrollo” para su revisión [1].

2.1. Primer sprint

En este primer sprint se establecieron las bases del proyecto, definiendo qué se iba a desarrollar, cómo se llevaría a cabo y en qué plazos se organizarían las distintas fases. Durante esta etapa se identificaron y planificaron todas las tareas necesarias para iniciar el proyecto.

Prácticamente todas las tareas definidas para este sprint fueron completadas, a excepción del diagrama de secuencia, cuyo motivo se explicará en apartados posteriores. Asimismo, el diseño de la base de datos no pudo completarse debido a la falta de tiempo, ya que las tareas de diseño y *sketching* requirieron más dedicación de la prevista. No obstante, la tarea fue finalizada durante el segundo sprint.

Tareas - Sprint 1



Actividad 1.1 - Definición del sistema

- ☒ ~~Tarea 1.1.1 - Definir objetivos~~
- ☒ ~~Tarea 1.1.2 - Estudio de mercado~~
- ☒ ~~Tarea 1.1.3 - Entrevistas iniciales~~

Actividad 1.2 - Análisis de requisitos

- ☒ ~~Tarea 1.2.1 - Requisitos funcionales~~
- ☒ ~~Tarea 1.2.2 - Requisitos no funcionales~~

Actividad 1.3 - Diseño del sistema

- ☒ ~~Tarea 1.3.1 - Identificación de actores~~
- ☒ ~~Tarea 1.3.2 - Diagrama de casos de uso~~
- ☒ ~~Tarea 1.3.3 - Definir tecnologías~~
- ☒ ~~Tarea 1.3.4 - Diseño de la arquitectura~~
- ☒ ~~Tarea 1.3.5 - Diagrama de clases~~
- ☐ Tarea 1.3.6 - Diagramas de secuencia
- ☐ Tarea 1.3.7 - Diseño de la BD

Actividad 1.4 - Diseño UX/UI

- ☒ ~~Tarea 1.4.1 - Sketch inicial~~
- ☒ ~~Tarea 1.4.2 - Prototipo high-fidelity~~

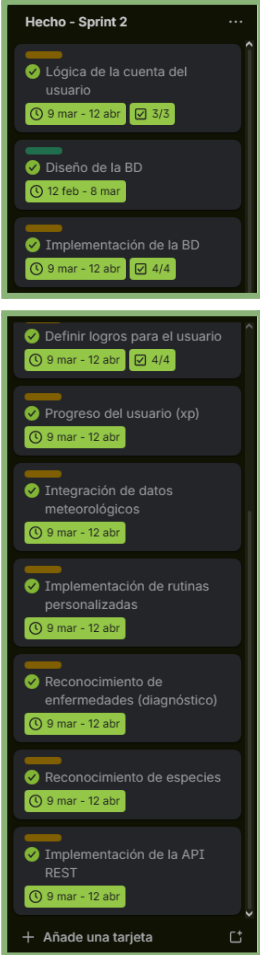
2.2. Segundo sprint

El segundo sprint se centró en el desarrollo del backend de la aplicación, incluyendo la integración de llamadas a APIs y la implementación de la lógica principal del sistema.

Entre las funcionalidades desarrolladas se encuentran la identificación de especies, la creación de rutinas personalizadas mediante inteligencia artificial y la incorporación de elementos de gamificación.

A diferencia del sprint anterior, en este caso todas las tareas fueron completadas, incluyendo el diseño de la base de datos, que había quedado pendiente.

Durante este sprint fue necesario consultar diversas referencias sobre la implementación de APIs y la conexión de *endpoints*, así como integrar todo este desarrollo dentro del repositorio del proyecto en GitHub.

Tareas - Sprint 2	
	Actividad 1.3 - Diseño del sistema ☒ Tarea 1.3.7 - Diseño de la BD Actividad 2.1 - Desarrollo del backend ☒ Tarea 2.1.1 - Implementación de la BD ☒ Tarea 2.1.2 - Implementación de la API REST Actividad 2.2 - Implementación del reconocimiento inteligente ☒ Tarea 2.2.1 - Reconocimiento de especies ☒ Tarea 2.2.2 - Reconocimiento de enfermedades ☒ Tarea 2.2.3 - Implementación de rutinas personalizadas ☒ Tarea 2.2.4 - Integración de datos meteorológicos Actividad 2.3 - Gamificación ☒ Tarea 2.3.1 - Progreso del usuario ☒ Tarea 2.3.2 - Definir logros para el usuario

3. Estado del proyecto

Teniendo en cuenta los puntos explicados anteriormente, el proyecto se encuentra terminando la fase de desarrollo del *backend*. La planificación inicial ya ha sido completada, por lo que en este momento se está siguiendo el *planning* previamente definido para garantizar el cumplimiento de los plazos establecidos.

En cuanto a la gestión del código, tal y como se ha mencionado, se está utilizando GitHub siguiendo la estrategia de una nueva rama por cada funcionalidad. Este enfoque permite organizar mejor el desarrollo y evitar conflictos durante la implementación de nuevas características.

En los siguientes apartados se explicará el trabajo realizado con más detalle pero, resumidamente, el *backend* ya incluye las llamadas a las APIs externas, así como la lógica relacionada con la gestión de usuarios y de plantas, de acuerdo con lo definido en los requisitos.

3.1. Documento de especificación del sistema

Durante la primera fase se decidió crear el **documento de especificación del sistema** debido a que apareció la necesidad de centralizar y organizar toda la información relacionada con la arquitectura y la definición técnica de la aplicación. Es un elemento clave dentro del desarrollo de Plant Guardian ya que funciona como una referencia para tomar decisiones al contener los requisitos y objetivos determinados.

Cabe destacar que se trata de un documento que está en constante evolución. Esto se debe a que la lógica asociada a las funcionalidades se va refinando progresivamente, ya sea optimizando su implementación o reduciendo su complejidad para ajustarse mejor al alcance del proyecto.

El documento incluye la definición de requisitos funcionales y no funcionales, la identificación de actores principales y secundarios, así como diagramas de casos de uso relacionados con la cuenta del usuario y la gestión de las plantas. Asimismo, incorpora el diseño del sistema mediante la descripción de la arquitectura, junto con diagramas de secuencia que detallan el comportamiento de las funcionalidades y el diseño de la base de datos [2].

En este informe no se mostrarán las pantallas desarrolladas, ya que se adjuntará el documento de especificaciones que recoge todas las pantallas relevantes, junto con los sketches iniciales creados. Además, algo a tener en cuenta es que durante su elaboración, se realizaron diversos cambios a lo largo del proceso de diseño.

En los apartados siguientes se detallarán los cambios que han surgido en el documento, junto con las decisiones tomadas respecto a ellos y los resultados obtenidos, mostrando cómo se ha adaptado el desarrollo de la aplicación a estas modificaciones.

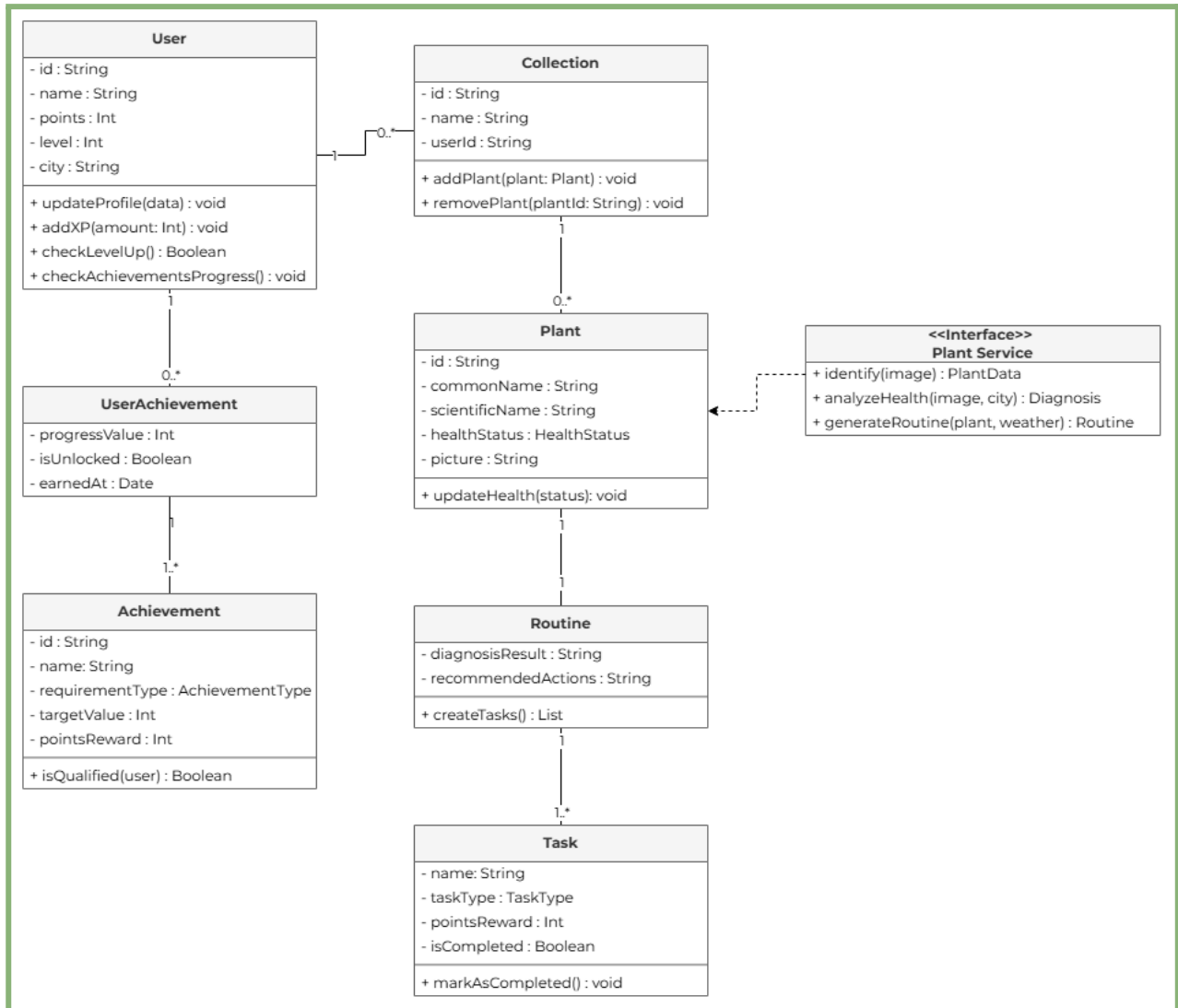
3.1.1. Cambios en los diagramas

Primero de todo, se tenía planificado incluir un diagrama de clases y otro de secuencia, pero finalmente se decidieron descartarlos por tiempo y por otras razones:

- **Diagrama de clases:** El backend de la aplicación está basado en llamadas a APIs y en una lógica centrada en ellas, por lo que el diagrama de clases no reflejaría fielmente la estructura real del sistema. A pesar de esto, se empezó una primera versión que actualmente está archivada.
- **Diagrama de secuencia:** Dentro de las sesiones para realizar el sketch y el prototipo de la aplicación se aclararon los flujos de lógica y las llamadas entre módulos. Además, los tipos de llamada iban a ser muy parecidos entre todos, siendo llamadas basadas a las APIs o a la base de datos para obtener información.

Cabe mencionar que el diagrama de clases se realizó en inglés debido a que en un inicio se había planteado codificar la aplicación en ese idioma. Sin embargo, para lograr un código más claro y comprensible, se optó por implementar la aplicación en español.

A pesar de no acabar de definir el diagrama de clases, éste terminó siendo útil como referencia para definir las tablas que se debían incluir en la base de datos, contribuyendo así a la organización y planificación de la información almacenada. La siguiente imagen corresponde al diagrama **descartado**.



Por último, cabe mencionar que en los diagramas de casos de uso se aplicaron pequeños cambios debido a la simplificación y cambios en la lógica de las APIs. También se añadieron las funcionalidades que faltaban según los requisitos acordados.

3.1.2. Estructura de la base de datos

Tal y como se ha mencionado en el anterior punto, se había previsto usar el diagrama de clases como referencia para la creación de la base de datos, pero finalmente se optó por construir una base inicial con datos básicos en cada tabla. No obstante, se mantuvo en cuenta la estructura de las clases creadas, que en gran medida sirvieron como guía para definir las tablas que conformarían la base de datos.

Una vez establecida esta base inicial, a medida que se desarrollaban las funciones y se revisaba el diseño, se fueron añadiendo los campos necesarios para asegurar que la

información almacenada fuera la correcta y suficiente. Siempre se priorizó incluir únicamente lo necesario, evitando sobrecargar la base de datos con información que pudiera no ser utilizada, garantizando así una estructura eficiente y coherente.

3.1.3. Arquitectura del sistema

La arquitectura del sistema fue modificada en relación con las APIs inicialmente previstas. En un primer momento se planteó utilizar la API Trefle para obtener información adicional sobre las plantas. Sin embargo, tras analizar su funcionamiento se observó que no aportaba información significativamente diferente a la que ya proporcionaban otras APIs utilizadas en el sistema.

En concreto, Pl@ntNet ya se encarga de procesar las imágenes y realizar la identificación de la especie de la planta, mientras que Gemini se utiliza para generar recomendaciones de cuidado, rutinas y tareas personalizadas para el usuario. Debido a esta superposición de funcionalidades, se decidió eliminar la integración con Trefle para simplificar la arquitectura y reducir dependencias externas innecesarias.

3.2. Definición de las funcionalidades

Para definir y concretar las funcionalidades del sistema, se llevaron a cabo entrevistas con usuarios, se elaboró un *sketch* inicial y se creó un prototipo en Figma de la aplicación. Estas herramientas permitieron identificar las necesidades reales de los usuarios y traducirlas en funcionalidades claras y prácticas para la aplicación.

3.2.1. Entrevistas a usuarios

Se realizaron tres entrevistas durante la primera fase del proyecto con el objetivo de comprender mejor las necesidades de los usuarios finales, definir con mayor precisión los requisitos de la aplicación y obtener una visión variada del cuidado de plantas en entornos reales. A continuación se explican las respuestas y conclusiones obtenidas.

Primero de todo, en cuanto a los principales problemas, los usuarios coinciden en la dificultad para determinar correctamente la frecuencia y cantidad de riego, así como en la identificación de plagas, enfermedades y condiciones adecuadas de luz o clima. Todo esto viene determinado a la falta de información y de asistencia según la planta.

Respecto al uso de tecnología, ninguno de los entrevistados utiliza actualmente aplicaciones especializadas, aunque todos muestran interés en su uso. Las funcionalidades más valoradas son la identificación de plantas mediante imagen, la generación de recordatorios de los cuidados y la obtención de información clara y útil sobre el mantenimiento de cada planta.

Por último, en términos de experiencia de usuario, se destaca la preferencia por aplicaciones intuitivas, con información clara y visual, evitando la sobrecarga de contenido. Además, se valora positivamente disponer de un resumen visual del estado general de las plantas.

Toda esta información obtenida permitió definir con mayor claridad cómo debía desarrollarse la aplicación, así como las funcionalidades prioritarias a implementar. En consecuencia, se establecieron como aspectos clave la simplicidad de uso, la claridad de la información mostrada y la incorporación de funcionalidades útiles como recordatorios de cuidado, identificación de plantas y recomendaciones personalizadas.

3.2.2. Diseño de la interfaz

Aunque los objetivos del proyecto se han mantenido, sí ha cambiado la perspectiva sobre cómo alcanzarlos. En un inicio, especialmente durante la elaboración del *sketch* de la aplicación, el flujo aún estaba poco definido, lo que generaba dudas sobre qué información debía mostrarse en cada pantalla.

Por este motivo, el *sketch* se creó con el objetivo de comprender mejor qué incluir, qué mostrar y cómo implementar las funcionalidades. Este proceso resultó especialmente complejo, ya que, teniendo en cuenta las respuestas obtenidas en las entrevistas, se buscó diseñar una aplicación lo más sencilla posible para evitar confusiones en el usuario.

Una vez obtenida una primera versión del *sketch*, se decidió avanzar hacia un prototipo de mayor calidad mediante la creación del diseño en Figma [5]. Esto permitió definir con mayor precisión el flujo de navegación de la aplicación. De hecho, se redujo el número de pantallas, eliminando aquellas que resultaban innecesarias o redundantes.

Además, en esta fase se tuvo presente el alcance del proyecto: desarrollar un MVP funcional. Por ello, se priorizó la implementación de los requisitos principales, manteniendo la posibilidad de incorporar los opcionales en el futuro. De esta manera, se evitaron complicaciones innecesarias y se logró un diseño de interfaz más limpio, claro y sencillo.

3.3. Construcción del backend

El *backend* se ha desarrollado utilizando FastAPI, un framework basado en Python que permite la gestión de operaciones asíncronas [3]. Esta arquitectura está dividida por capas (*layered architecture*) [4] y se basa fundamentalmente en el **principio de separación de responsabilidades** [5]. Las tres capas principales son:

- **Routers:** Definen los *endpoints* de la API (clima, plantas, tareas y usuarios) y gestionan el flujo de peticiones HTTP.
- **Lógica de negocio:** Contiene el núcleo funcional de la aplicación, como el cálculo de experiencia, la validación de logros y el procesamiento de tareas.
- **Servicios externos:** Módulos especializados en la comunicación con APIs de terceros (Gemini, OpenWeather y PlantNet).

El flujo general de funcionamiento mediante las tres capas mencionadas sigue la siguiente estructura:

ROUTER (*endpoint*) → LOGIC (procesamiento) → SERVICE (llamada a la API)

3.3.1. Agregación de las APIs

La integración de servicios de terceros se ha centralizado en una misma carpeta, siguiendo el patrón de diseño de **encapsulamiento** [6]. Esto permite que el resto de la aplicación interactúe con las APIs externas de forma simplificada a través de funciones asíncronas, sin necesidad de conocer los detalles técnicos de cada proveedor.

Para la identificación y análisis de plantas, se utilizan las APIs de Pl@ntNet y Google Gemini. Primero, Pl@ntNet identifica la especie a partir de la imagen proporcionada por el usuario [7]. Posteriormente, Gemini analiza esa información junto con la imagen para generar un diagnóstico y recomendaciones de cuidado [8]. Las respuestas de la IA se procesan y se convierten en estructuras JSON, lo que permite almacenarlas de forma organizada en la base de datos.

En cuanto al contexto climático, se integra la API de OpenWeather, que proporciona datos meteorológicos en función de la ubicación del usuario [9]. El *backend* procesa esta información y devuelve una versión simplificada al *frontend*, permitiendo mostrar datos relevantes para el cuidado de las plantas.

Por último, la gestión de archivos multimedia se realiza mediante Supabase Storage, donde se almacenan las imágenes de las plantas y otros recursos. El sistema incluye funciones para la subida de imágenes con identificadores únicos (UUID), la generación de URLs públicas para su visualización y la eliminación automática de archivos cuando se borra el registro correspondiente en la base de datos.

Todas estas integraciones se exponen mediante *endpoints* del *backend* documentados automáticamente por FastAPI, permitiendo que la aplicación móvil realice peticiones simples y reciba respuestas ya procesadas por el sistema.

3.3.2. Gamificación

El sistema de gamificación de la aplicación se ha diseñado con el objetivo de motivar al usuario a cuidar sus plantas de forma constante y fomentar la participación dentro de la app. Para ello, se han incorporado distintos mecanismos que recompensan el progreso y la actividad del usuario. Los componentes principales son los siguientes:

- **Sistema de progresión:** Algoritmo de subida de nivel con una curva de dificultad dinámica (multiplicador de 1.2x).
- **Motor de logros:** Un verificador automatizado que otorga medallas en tres categorías diferentes, que son el nivel alcanzado, la cantidad de plantas registradas y las tareas completadas.
- **Contadores acumulativos:** Se ha implementado un sistema de persistencia en la tabla *Usuario* para registrar las tareas completadas “de por vida”, permitiendo que los logros de constancia sean independientes del reseteo de tareas diarias.
- **Iconos de usuario desbloqueables:** Lógica que vincula el nivel del usuario con la disponibilidad de iconos según su nivel, permitiendo al usuario cambiar su avatar únicamente por aquellos que ha desbloqueado al tener el nivel requerido.

3.3.3. Librerías

Dentro del código se utilizan varias librerías de Python, cada una con un propósito específico dentro del proyecto:

- Validación de datos:
 - **Uvicorn** actúa como el servidor de ejecución, manteniendo la aplicación en funcionamiento y escuchando las peticiones en tiempo real [10].
 - **Pydantic** asegura que los datos que recibe el servidor cumplan con la estructura esperada, evitando errores y garantizando que la información procesada y almacenada sea correcta [11].
- Seguridad y autenticación de los usuarios:
 - **Bcrypt** se utiliza para proteger las contraseñas mediante un algoritmo de cifrado seguro (*hashing*) [12].
 - **PyJWT** gestiona los tokens de sesión, permitiendo que los usuarios inicien sesión una vez y sean reconocidos en sus acciones posteriores [13].
- Comunicación con servicios externos:
 - **HTTPX** es un cliente de red que permite al *backend* enviar y recibir información desde APIs externas [14].
- Manejo de datos únicos y procesamiento de información:
 - **UUID** genera identificadores únicos para cada usuario o planta, evitando duplicidades en la base de datos [15].
 - **RE** permite buscar y extraer información específica de textos [16].
 - **Datetime** para las fechas y tiempos, calculando vencimientos de tareas como riego o fertilización de forma automática [17].

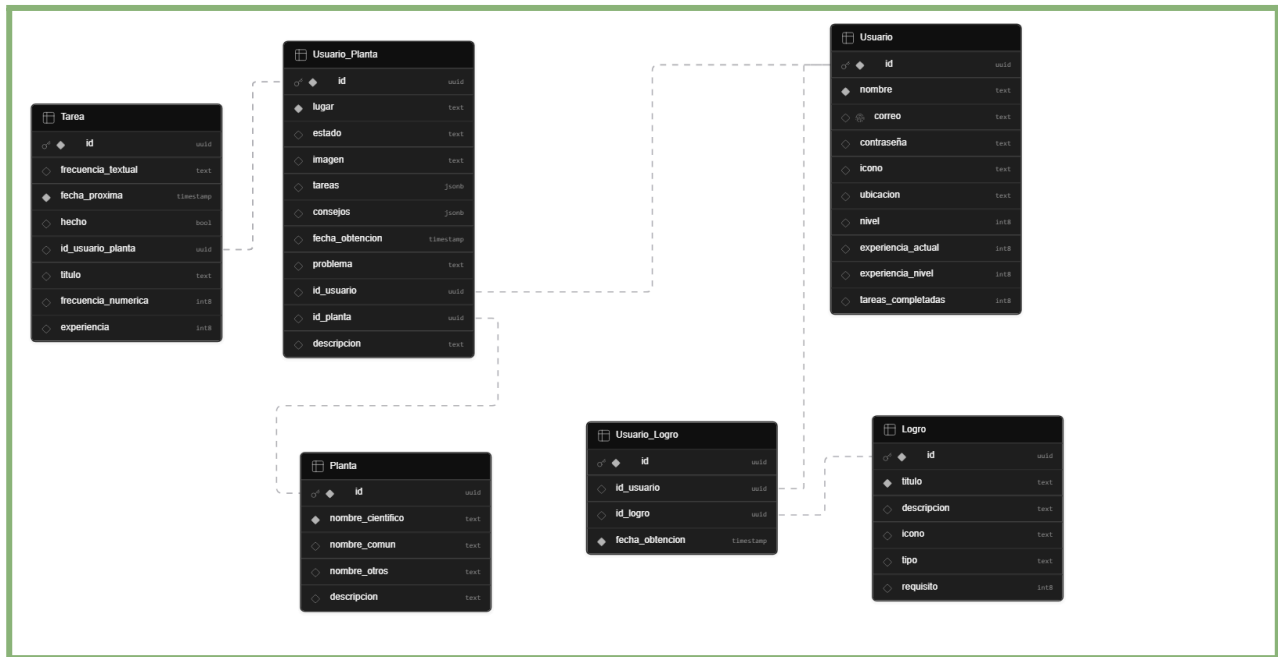
3.4. Base de datos

La aplicación utiliza Supabase, basado en PostgreSQL, como sistema de gestión de base de datos relacional. El diseño del esquema de la base de datos (ERD) se ha planteado con el objetivo de garantizar la integridad de los datos, mantener una estructura clara entre entidades y asegurar una buena eficiencia en las consultas [18].

Para lograrlo, se han definido las siguientes características principales:

- **Entidades principales:** Tablas de *Usuario*, *Planta*, *Tarea* y *Logro*.
- **Tablas de relación:** Uso de *Usuario_Planta* para gestionar las instancias específicas de cada planta y *Usuario_Logro* como registro de hitos alcanzados.
- **Optimización de tipos:** Empleo de UUIDs para identificadores únicos y tipos JSONB para almacenar estructuras de datos flexibles [19], como los consejos de la IA, para evitar crear tablas o estructuras extras.
- **Seguridad:** Configuración de políticas de RLS (*Row Level Security*) para asegurar que cada usuario solo pueda acceder y modificar sus propios datos [20].

En la siguiente imagen se puede observar la estructura de la base de datos, creada en Supabase, donde se aprecian las distintas tablas y las relaciones entre ellas.



4. Inteligencia artificial

En el desarrollo del proyecto se ha utilizado principalmente Gemini como herramienta de apoyo. Su uso ha servido como orientación en la organización del código, facilitando la comprensión de las conexiones entre los distintos componentes y la forma en que debían implementarse.

Asimismo, ha servido como guía en el diseño de la base de datos, ayudando a estructurar las tablas y sus relaciones de manera más eficiente. También se ha empleado para resolver dudas relacionadas con las funcionalidades de la aplicación, como la información que debía mostrarse al usuario o la definición de elementos de gamificación, como logros.

Gracias a los ejemplos de código proporcionados, ha sido posible adaptarse rápidamente al entorno de desarrollo, especialmente en las fases iniciales, donde se realizaron numerosas consultas sobre las tecnologías que se podrían implementar.

Cabe destacar que se exploraron diferentes propuestas de organización del backend, pero finalmente se optó por una estructura similar al modelo-vista-controlador (MVC), debido a su familiaridad y claridad.

En conclusión, el uso de la inteligencia artificial ha facilitado la comprensión global del sistema y ha contribuido a agilizar el desarrollo del proyecto. Además, ha permitido reducir el tiempo de aprendizaje en tecnologías y conceptos desconocidos, ofreciendo ejemplos prácticos y soluciones iniciales sobre las que trabajar.

5. Webgrafía

- [1] *Cómo implementar la metodología ágil Scrum en Trello para tu equipo de desarrollo y Marketing* | by Jorge Iván Grisales Marín | Medium. (s. f.). Recuperado 1 de marzo de 2026, de <https://medium.com/@jivagrisma/c%C3%B3mo-implementar-la-metodolog%C3%ADa-%C3%A1gil-scrum-en-trello-para-tu-equipo-de-desarrollo-y-marketing-c230bdebe87>
- [2] *Documentos de Especificación Funcional: tu guía completa*. (s. f.). Recuperado 1 de abril de 2026, de <https://www.justinmind.com/es/blog/especificacion-funcional-documentacion/>
- [3] *Features - FastAPI*. (s. f.). Recuperado 9 de marzo de 2026, de <https://fastapi.tiangolo.com/features/#automatic-docs>
- [4] *Arquitecturas de aplicaciones web comunes - .NET* | Microsoft Learn. (s. f.). Recuperado 10 de marzo de 2026, de <https://learn.microsoft.com/es-es/dotnet/architecture/modern-web-apps-azure/common-web-application-architectures>
- [5] *Principios arquitectónicos - .NET* | Microsoft Learn. (s. f.). Recuperado 10 de marzo de 2026, de <https://learn.microsoft.com/es-es/dotnet/architecture/modern-web-apps-azure/architectural-principles>
- [6] *Encapsulación en la programación orientada a objetos*. (s. f.). Recuperado 14 de marzo de 2026, de <https://www.cursosdesarrolloweb.es/blog/encapsulacion-programacion-orientada-objetos>
- [7] *Introduction - Pl@ntNet API for developers*. (s. f.). Recuperado 11 de marzo de 2026, de <https://my.plantnet.org/doc/getting-started/introduction>
- [8] *Gemini API | Google AI for Developers*. (s. f.). Recuperado 11 de marzo de 2026, de <https://ai.google.dev/gemini-api/docs?hl=es-419>
- [9] *OpenWeatherMap API guide*. (s. f.). Recuperado 18 de marzo de 2026, de <https://openweathermap.org/guide>
- [10] *Uvicorn*. (s. f.). Recuperado 8 de marzo de 2026, de <https://uvicorn.dev/>
- [11] *Welcome to Pydantic - Pydantic Validation*. (s. f.). Recuperado 8 de marzo de 2026, de <https://docs.pydantic.dev/latest/>
- [12] *bcrypt · PyPI*. (s. f.). Recuperado 15 de marzo de 2026, de <https://pypi.org/project/bcrypt/>
- [13] *PyJWT · PyPI*. (s. f.). Recuperado 10 de marzo de 2026, de <https://pypi.org/project/PyJWT/>

- [14] *HTTPX*. (s. f.). Recuperado 10 de marzo de 2026, de <https://www.python-httpx.org/>
- [15] *uuid — UUID objects according to RFC 9562 — Python 3.14.4 documentation*. (s. f.). Recuperado 13 de marzo de 2026, de <https://docs.python.org/3/library/uuid.html>
- [16] *re — Regular expression operations — documentación de Python - 3.14.4*. (s. f.). Recuperado 14 de marzo de 2026, de <https://docs.python.org/es/3/library/re.html>
- [17] *datetime — Basic date and time types — documentación de Python - 3.14.4*. (s. f.). Recuperado 14 de marzo de 2026, de <https://docs.python.org/es/3/library/datetime.html>
- [18] *Diseñando una base de datos en el modelo relacional | campusMVP.es*. (s. f.). Recuperado 9 de marzo de 2026, de https://www.campusmvp.es/recursos/post/Disenando-una-base-de-datos-en-el-modelo-relacional.aspx?srsId=AfmBOoo7-Z3mUO8hAgajYcX6VK5n1AU_S85EJC7BDTi3B94ZWp3VRTyh
- [19] *Trabaja con datos JSONB | Spanner | Google Cloud Documentation*. (s. f.). Recuperado 9 de marzo de 2026, de <https://docs.cloud.google.com/spanner/docs/working-with-jsonb?hl=es-419>
- [20] *Seguridad de nivel de fila (RLS) con Power BI - Microsoft Fabric | Microsoft Learn*. (s. f.). Recuperado 16 de marzo de 2026, de <https://learn.microsoft.com/es-es/fabric/security/service-admin-row-level-security>